

✓ Schenider ml explainability

```
from google.colab import drive
```

```
import pandas as pd
import numpy as np
```

```
path_csv = "/content/drive/MyDrive/Dathaton_2025/Data/dataset.csv"
pd_data = pd.read_csv(path_csv)
```

```
pd_data.head()
```

	id	product_A_sold_in_the_past	product_B_sold_in_the_past	product_A_recommended	product_A	product_C	product_D
0	1	-0.25992	-0.34794	-0.1097	-0.0891	-0.02372	-0.02372
1	2	-0.25992	-0.28599	-0.1097	-0.0891	-0.02372	-0.02372
2	3	-0.25992	-0.18998	-0.1097	-0.0891	-0.02372	-0.02372
3	4	0.65945	3.35306	-0.1097	-0.0891	-0.02372	-0.02372
4	5	-0.25992	-0.24623	-0.1097	-0.0891	-0.02372	-0.02372

Data exploration Schneider

✓ Task

Perform an initial data exploration on the `pd_data` DataFrame by checking for missing values, visualizing the distributions of key numerical columns using histograms or box plots, and generating a correlation matrix heatmap to understand feature relationships, then provide a summary of the findings.

✓ Check for Missing Values

Subtask:

Identify and display the count of missing (NaN) values for each column in the `pd_data` DataFrame.

Reasoning: To identify and display the count of missing values for each column, I will use the `.isnull()` method to check for NaN values and then `.sum()` to get the total count for each column in the `pd_data` DataFrame.

```
missing_values_count = pd_data.isnull().sum()
print("Missing values count per column:")
print(missing_values_count)
```

```
Missing values count per column:
id                0
product_A_sold_in_the_past  0
product_B_sold_in_the_past  0
product_A_recommended    0
product_A         0
product_C         0
product_D         0
cust_hitrate       0
cust_interactions  0
cust_contracts     0
opp_month          0
opp_old           0
```

```
competitor_Z      0
competitor_X      0
competitor_Y      0
cust_in_iberia    0
target_variable    0
dtype: int64
```

Visualize Data Distributions

Subtask:

Create visualizations, such as histograms or box plots, for key numerical columns in `pd_data` to understand their distributions and identify potential outliers.

Reasoning: Before visualizing, I need to import the `matplotlib.pyplot` and `seaborn` libraries to enable plotting functionalities as specified in the instructions. This is a foundational step for creating visualizations.

```
import matplotlib.pyplot as plt
import seaborn as sns
print("Libraries matplotlib.pyplot and seaborn imported successfully.")
```

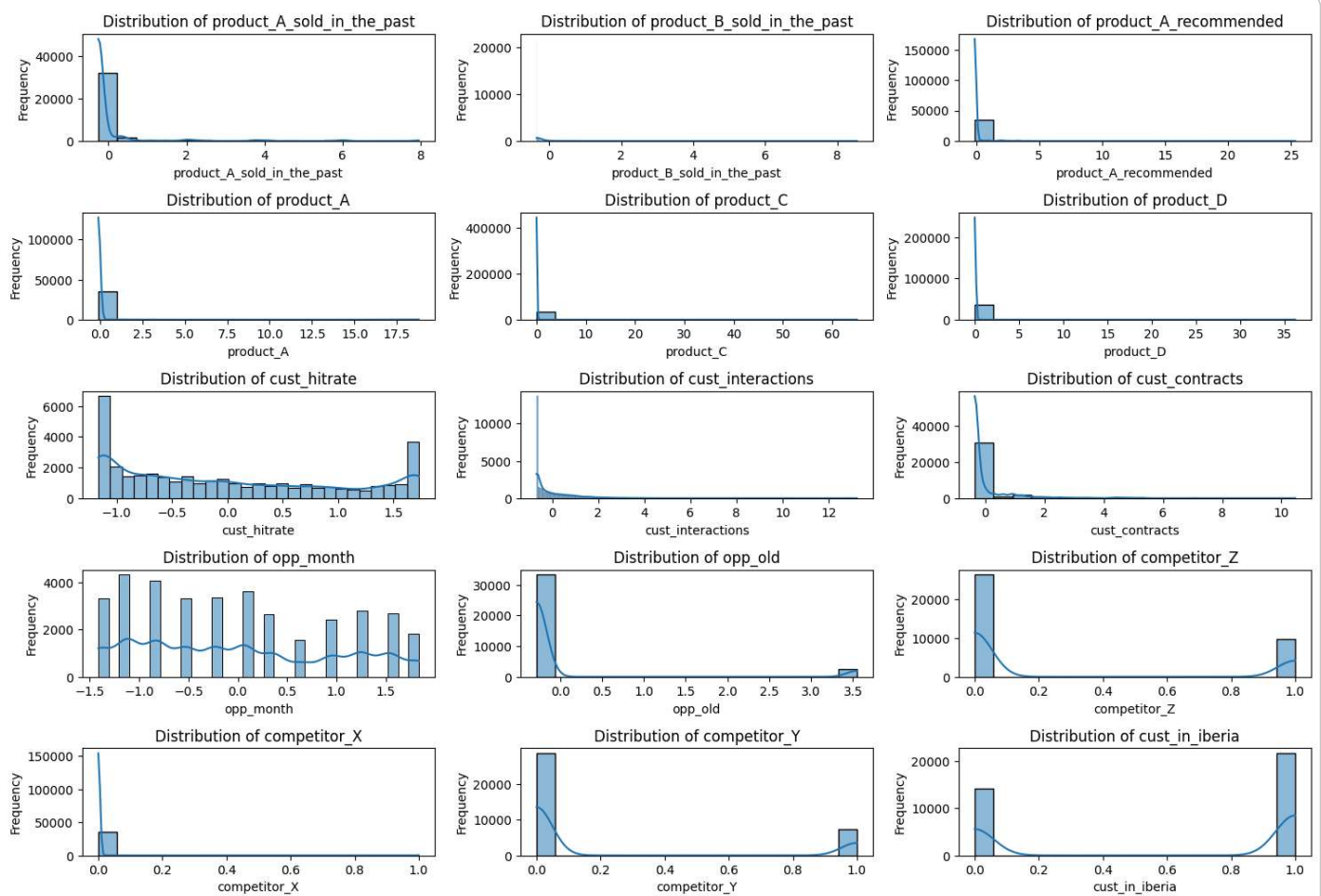
```
Libraries matplotlib.pyplot and seaborn imported successfully.
```

Reasoning: To visualize the distributions of numerical columns, I will first identify them, excluding 'id' and 'target_variable', then iterate through these columns to create and display histograms using seaborn, ensuring each plot has a descriptive title.

```
numerical_cols = pd_data.select_dtypes(include=np.number).columns.tolist()
# Exclude 'id' and 'target_variable' if they are present
if 'id' in numerical_cols:
    numerical_cols.remove('id')
if 'target_variable' in numerical_cols:
    numerical_cols.remove('target_variable')

plt.figure(figsize=(15, 2 * len(numerical_cols)))
for i, col in enumerate(numerical_cols):
    plt.subplot(len(numerical_cols), 3, i + 1) # Adjust subplot grid as needed
    sns.histplot(pd_data[col], kde=True)
    plt.title(f'Distribution of {col}')
    plt.xlabel(col)
    plt.ylabel('Frequency')
plt.tight_layout()
plt.show()

print("Histograms for numerical columns displayed successfully.")
```



▼ Visualize Feature Relationships

Subtask:

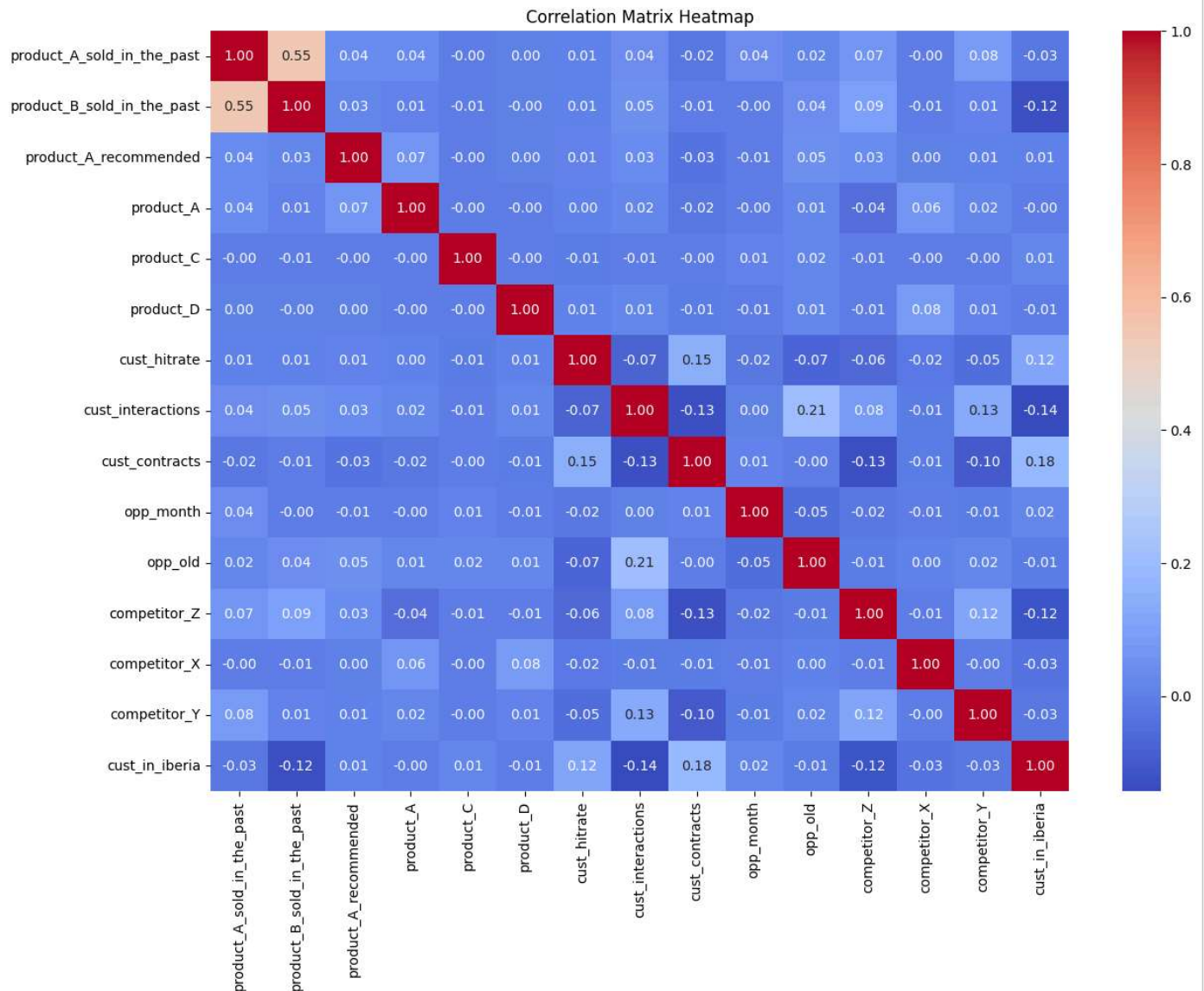
Generate a correlation matrix heatmap to visualize the relationships between numerical features in `pd_data`.

Reasoning: To visualize feature relationships, I will calculate the correlation matrix for the numerical columns, then generate a heatmap using seaborn, ensuring it's readable with annotations and a title.

```
correlation_matrix = pd_data[numerical_cols].corr()

plt.figure(figsize=(14, 10))
sns.heatmap(correlation_matrix, annot=True, cmap='coolwarm', fmt=".2f")
plt.title('Correlation Matrix Heatmap')
plt.show()

print("Correlation matrix heatmap displayed successfully.")
```



Correlation matrix heatmap displayed successfully.

Summary of Findings

Missing Values:

Upon checking for missing values, it was found that the `pd_data` DataFrame contains **no missing values** across any of its columns. This is a positive finding as it means no imputation or special handling for missing data is required at this stage.

Data Distributions:

The histograms for the numerical columns provided insights into their distributions:

- Most numerical features appear to have distributions that are somewhat centered, though many exhibit skewness (e.g., `product_B_sold_in_the_past`, `cust_interactions`, `opp_month`).
- Several columns, such as `competitor_Z`, `competitor_X`, `competitor_Y`, and `cust_in_iberia`, are binary or categorical, as indicated by their distributions showing only two distinct bars.
- Some features like `product_A_sold_in_the_past` and `product_B_sold_in_the_past` show a concentration of values at the lower end with some outliers or higher values.

Feature Relationships (Correlation Matrix Heatmap):

- The heatmap revealed various levels of correlation between numerical features.
- Strong positive correlations were observed between `product_A_sold_in_the_past` and `product_B_sold_in_the_past`, suggesting that these product sales might be related.
- `cust_hitrate`, `cust_interactions`, and `cust_contracts` show some positive correlations among themselves, which is expected as they are all related to customer engagement.
- `opp_month` and `opp_old` do not appear to have strong correlations with most other features.

- Binary features like `competitor_Z`, `competitor_X`, `competitor_Y`, and `cust_in_iberia` also show their relationships with other continuous variables, with some minor correlations observed. For example, `cust_in_iberia` shows a slight positive correlation with `product_A_sold_in_the_past` and `product_B_sold_in_the_past`.
- The `target_variable` was excluded from this correlation analysis as it's typically treated as the dependent variable and not part of the independent feature relationships for initial exploration in this context.

Overall, this initial exploration provides a solid foundation for further analysis, model building, and feature engineering, highlighting data cleanliness and key relationships.

Summary:

Data Analysis Key Findings

- The `pd_data` DataFrame contains **no missing values** across any of its columns, simplifying further analysis.
- Most numerical features display skewed distributions (e.g., `product_B_sold_in_the_past`, `cust_interactions`, `opp_month`).
- Several columns, including `competitor_Z`, `competitor_X`, `competitor_Y`, and `cust_in_iberia`, are binary or categorical, indicated by their distinct two-bar distributions.
- Strong positive correlations were observed between `product_A_sold_in_the_past` and `product_B_sold_in_the_past`.